

FUNDAMENTOS DE LENGUAJE ENSAMBLADOR 2

DEFINICIÓN DE DATOS

■ Tipos de Datos Intrínsecos

- MASM define tipos de datos intrínsecos, cada uno de los cuales describe un conjunto de valores que pueden asignarse a las variables y expresiones del tipo dado.
- La característica esencial de cada tipo es su tamaño en bits: 8, 16, 32, 48, 64, 80.
- Las demás características (Por ejemplo: con signo, apuntador o de punto flotante) son opcionales y su propósito principal es para beneficio de los programadores que desean se les recuerde acerca del tipo de datos que guarda la variable.

Tipo	Uso
BYTE	Entero de 8 bits sin signo
SBYTE	Entero de 8 bits con signo
WORD	Entero de 16 bits sin signo [también puede ser un apuntador cercano (near) en modo de direccionamiento real]
SWORD	Entero de 16 bits con signo
DWORD	Entero de 32 bits sin signo [también puede ser un apuntador cercano (near) en modo protegido]
SDWORD	Entero de 32 bits con signo
FWORD	Entero de 48 bits [Apuntador lejano (far) en modo protegido]
QWORD	Entero de 64 bits
TBYTE	Entero de 80 bits (10 bytes)
REAL4	Número real corto IEEE de 32 bits (4 bytes)
REAL8	Número real largo IEEE de 64 bits (8 bytes)
REAL10	Número real extendido IEEE de 80 bits (10 bytes)

INSTRUCCIÓN DE DEFINICIÓN DE DATOS

- Una instrucción o estatuto de definición de datos separa espacio de almacenamiento en memoria para una variable, con un nombre opcional.
- Una definición de datos tiene la siguiente sintaxis:

[nombre] directiva inicializador [,inicializador].....
- Nombre: El nombre opcional que se asigna a una variable debe apegarse a las reglas para los identificadores.
- Directiva: La directiva en una instrucción de definición de datos puede ser BYTE,WORD,DWORD,SBYTE,SWORD o cualquiera de los tipos de la tabla anterior. Aemas, puede ser cualquiera de las directivas de definición de datos heredadas.

Directiva	Uso
DB	Entero de 8 bits
DW	Entero de 16 bits
DD	Entero o real de 32 bits
DQ	Entero o real de 64 bits
DT	Define 80 bits, o diez bytes

- 
- Inicializador. En una definición de datos se requiere por lo menos un inicializador, aunque sea cero. Los inicializadores adicionales (si los hay) van separados por comas.
 - Si se prefiere dejar las variables sin inicializar, se puede utilizar el símbolo ? Como inicializador.

DEFINICIÓN DE DATOS BYTE Y SBYTE

- Las directivas BYTE (definir byte) y SBYTE (definir byte con signo) asignan espacio de almacenamiento para uno o más valores con o sin signo. Cada inicializador debe caber en 8 bits de almacenamiento.

<code>valor1</code>	<code>BYTE</code>	<code>'A'</code>	<code>; constante tipo carácter</code>
<code>valor2</code>	<code>BYTE</code>	<code>0</code>	<code>; el byte sin signo más pequeño</code>
<code>valor3</code>	<code>BYTE</code>	<code>255</code>	<code>; el byte sin signo más grande</code>
<code>valor4</code>	<code>SBYTE</code>	<code>-128</code>	<code>; el byte con signo más pequeño</code>
<code>valor5</code>	<code>SBYTE</code>	<code>+127</code>	<code>; el byte con signo más grande</code>

- Un inicializador de signo de interrogación (?) deja la variable sin inicializar, indicando que se le asignará un valor en tiempo de ejecución:

`valor6` `BYTE` `?`

- El nombre opcional es una etiqueta que marca el desplazamiento de la variable, desde el inicio de su segmento. Por ejemplo, si **valor1** se encuentra en el desplazamiento 0000 dentro del segmento de datos y consume 1 byte de almacenamiento, **valor2** se encuentra de manera automática en el desplazamiento 0001:

```
valor1 BYTE 10h  
valor2 BYTE 20h
```

- La directiva heredada DB puede definir también una variable de 8 bits, con o sin signo:

```
val1 DB 255 ; byte sin signo  
val2 DB -128 ; byte con signo
```


MÚLTIPLES INICIALIZADORES

Si se utilizan varios inicializadores en la misma definición de datos, su etiqueta sólo hace referencia al desplazamiento del primer inicializador. En el siguiente ejemplo, asuma que **lista** se encuentra en el desplazamiento 0000. Si es así, entonces el valor 10 se encuentra en el desplazamiento 0000, 20 en el desplazamiento 0001, 30 en el desplazamiento 0002, y 40 en el desplazamiento 0003:.

lista BYTE 10,20,30,40

Desplazamiento	Valor
0000:	10
0001:	20
0002:	30
0003:	40

- Dentro de una definición de datos individual, sus inicializadores pueden utilizar distintas raíces. Pueden mezclarse libremente las constantes tipo carácter y de cadena. En el siguiente ejemplo, **lista1** y **lista2** tienen el mismo contenido

```
lista1 BYTE 10, 32, 41h, 00100010b  
lista2 BYTE 0Ah, 20h, 'A', 22h
```

DEFINICIÓN DE CADENAS

Para definir una cadena de caracteres, hay que encerrarlos entre comillas sencillas o dobles. El tipo más común de cadena termina con un byte nulo (que contiene 0).

```
saludo1 BYTE "Buenas tardes",0  
saludo2 BYTE 'Buenas noches',0
```

- Cada carácter utiliza un byte de almacenamiento. Las cadenas son una excepción a la regla que establece que los valores de bytes deben separarse por comas. Sin esta excepción, tendríamos que definir a **saludo1** de la siguiente manera:

```
saludo1 BYTE 'B','u','e','n','a','s'....etc.
```

Una cadena puede distribuirse a través de varias líneas, sin tener que suministrar una etiqueta para cada línea:

```
saludo1 BYTE "Bienvenido al programa de demostración de Cifrado "  
        BYTE "creado por Kip Irvine.",0dh,0ah  
        BYTE "Si desea modificar este programa, por favor "  
        BYTE "envíeme una copia.",0dh,0ah,0
```

Los códigos hexadecimales 0Dh y 0Ah se llaman también CR/LF (retorno de carro/avance de línea) o *caracteres de fin de línea*. Cuando se escriben en la salida estándar, desplazan el cursor hacia la columna izquierda de la línea que sigue a la línea actual.

DEFINICIÓN DE DATOS WORD Y SWORD

- Las directivas WORD (definir palabra) y SWORD (*definir palabra con signo*) crean almacenamiento para uno o más enteros de 16 bits:

```
palabra1 WORD 65535           ; el valor sin signo más grande
palabra2 SWORD -32768         ; el valor con signo más pequeño
palabra3 WORD ?               ; sin inicializar, sin signo
```

- También puede usarse la directiva DW heredada:

```
val1 DW 65535                 ; sin signo
val2 DW -32768                 ; con signo
```

ARREGLOS DE PALABRAS

- Para crear un arreglo de palabras se listan los elementos o se usa el operador DUP. El siguiente arreglo contiene una lista de valores:

`miLista WORD 1,2,3,4,5`

- A continuación, se muestra un diagrama del arreglo en memoria, suponiendo que **miLista** empieza en el desplazamiento 0000. Las direcciones se incrementan en 2, ya que cada valor ocupa 2 bytes:

Desplazamiento	Valor
0000:	1
0002:	2
0004:	3
0006:	4
0008:	5

DEFINICIÓN DE DATOS DWORD Y SDWORD

- Las directivas DWORD (definir doble palabra) y SDWORD (definir doble palabra con signo) asignan almacenamiento para uno o más enteros de 32 bits:

```
val1 DWORD    12345678h           ; sin signo
val2 SDWORD   -2147483648         ; con signo
val3 DWORD    20 DUP(?)           ; arreglo sin signo
```

- También puede usarse la directiva DD heredada:

```
val1 DD 12345678h                 ; sin signo
val2 DD -2147483648               ; con signo
```

ARREGLOS DE DOBLES PALABRAS

- Para crear un arreglo de doubles palabras, se inicializa cada elemento en forma explícita, o se utiliza el operador DUP. He aquí un arreglo que contiene valores sin signo específicos:

```
miLista  DWORD 1,2,3,4,5
```

- A continuación, se muestra un diagrama del arreglo en memoria, suponiendo que **miLista** empieza en desplazamiento 0000. Los desplazamientos se incrementan por 4:

Desplazamiento	Valores
0000:	1
0004:	2
0008:	3
000C:	4
0010:	5

DEFINICIÓN DE DATOS QWORD

- La directiva QWORD (define palabra cuádruple) asigna almacenamiento para valores de 64 bits (8 bytes):

```
quad1 QWORD 1234567812345678h
```

- También puede usarse la directiva DQ heredada:

```
quad1 DQ 1234567812345678h
```

DEFINICIÓN DE DATOS TBYTE

- La directiva TBYTE (define diez bytes) crea almacenamiento para los enteros de 80 bits. Este tipo de datos se utiliza principalmente para almacenar números decimales codificados en binario. Para manipular estos valores se requieren instrucciones especiales en el conjunto de instrucciones de punto flotante:

```
val1 TBYTE 10000000000123456789Ah
```

- También puede utilizarse la directiva DT heredada:

```
val1 DT 10000000000123456789Ah
```

DEFINICIÓN DE DATOS DE NÚMEROS REALES

- REAL4 define a una variable real de 4 bytes y precisión simple. REAL8 define a un real de 8 bytes y precisión doble, y REAL10 define a un real de 10 bytes y doble precisión extendida. Cada uno requiere de uno o más inicializadores de constantes reales:

```
rVal1    REAL4    -1.2
rVal2    REAL8     3.2E-260
rVal3    REAL10   4.6E+4096
rArreglo REAL4    20 DUP(0.0)
```

LA SIGUIENTE TABLA DESCRIBE A CADA UNO DE LOS TIPOS REALES ESTÁNDAR, EN TÉRMINOS DE SU NÚMERO MÍNIMO DE DÍGITOS SIGNIFICATIVOS Y EL RANGO APROXIMADO:

Tipo de datos	Dígitos significativos	Rango aproximado
Real corto	6	1.18×10^{-38} a 3.40×10^{38}
Real largo	15	2.23×10^{-308} a 1.79×10^{308}
Real de precisión extendida	19	3.37×10^{-4932} a 1.18×10^{4932}

LAS DIRECTIVAS DD, DQ Y DT HEREDADAS PUEDEN DEFINIR NÚMEROS REALES:

```
rVal1 DD -1.2 ; real corto  
rVal2 DQ 3.2E-260 ; real largo  
rVal3 DT 4.6E+4096 ; real de precisión extendida
```

ORDEN LITTLE ENDIAN

0000:	78
0001:	56
0002:	34
0003:	12

Little endian

- Los procesadores Intel almacenan y recuperan datos de la memoria usando el orden *little endian*. El byte menos significativo se almacena en la primera dirección de memoria asignada para los datos. El resto de los bytes se almacenan en las siguientes posiciones consecutivas de memoria.
- Como ejemplo, considere la doble palabra 12345678h. Si se coloca en el desplazamiento 0000 de la memoria, se almacenaría 78h en el primer byte, 56h en el segundo byte, y el resto de los bytes estarían en los desplazamientos 0003 y 0004:

0000:	12
0001:	34
0002:	56
0003:	78

Big endian

- Hay otros sistemas computacionales que utilizan el orden *big endian* (de mayor a menor). La siguiente figura muestra un ejemplo del número 12345678h almacenado en orden big endian, en el desplazamiento 0:

DECLARACIÓN DE DATOS SIN INICIALIZAR

- La directiva `.DATA?` declara los datos sin inicializar. Al definir un bloque extenso de datos sin inicializar, la directiva `.DATA?` reduce el tamaño de un programa compilado. Por ejemplo, el siguiente código se declara de manera eficiente:

```
.data
arregloPequeno DWORD 10 DUP(0)      ; 40 bytes
.data?
arregloGrande  DWORD 5000 DUP(?)     ; 20,000 bytes sin inicializar
```

- Por otro lado, el siguiente código produce un programa compilado que es 20,000 bytes más grande:

```
.data
arregloPequeno DWORD 10 DUP(0)      ; 40 bytes
arregloGrande  DWORD 5000 DUP(?)     ; 20,000 bytes
```


MEZCLA DE CÓDIGO Y DATOS

```
.code
mov eax,ebx
.data
temp DWORD ?
.code
mov temp,eax
. . .
```

- El ensamblador nos permite cambiar entre el código y los datos en nuestros programas. Por ejemplo, tal vez podríamos llegar a necesitar declarar una variable que se utilice sólo dentro de un área localizada de un programa.
- El siguiente ejemplo inserta una variable llamada **temp** entre dos instrucciones de código:
- Aunque **temp** parece interrumpir el flujo de las instrucciones ejecutables, MASM coloca a **temp** en el segmento de datos, separada del segmento que guarda el código compilado.

```

.data segment
;add your data here!
var1 dw 10000
var2 dw 10000
var3 dw 10000
var4 10000 dw ;instruccion incorrecta
varFinal dw ?
lista BYTE 10,20,30,40
lista byte 0ah,20h,22h

ends

.stack segment
dw 128 dup(0)
ends

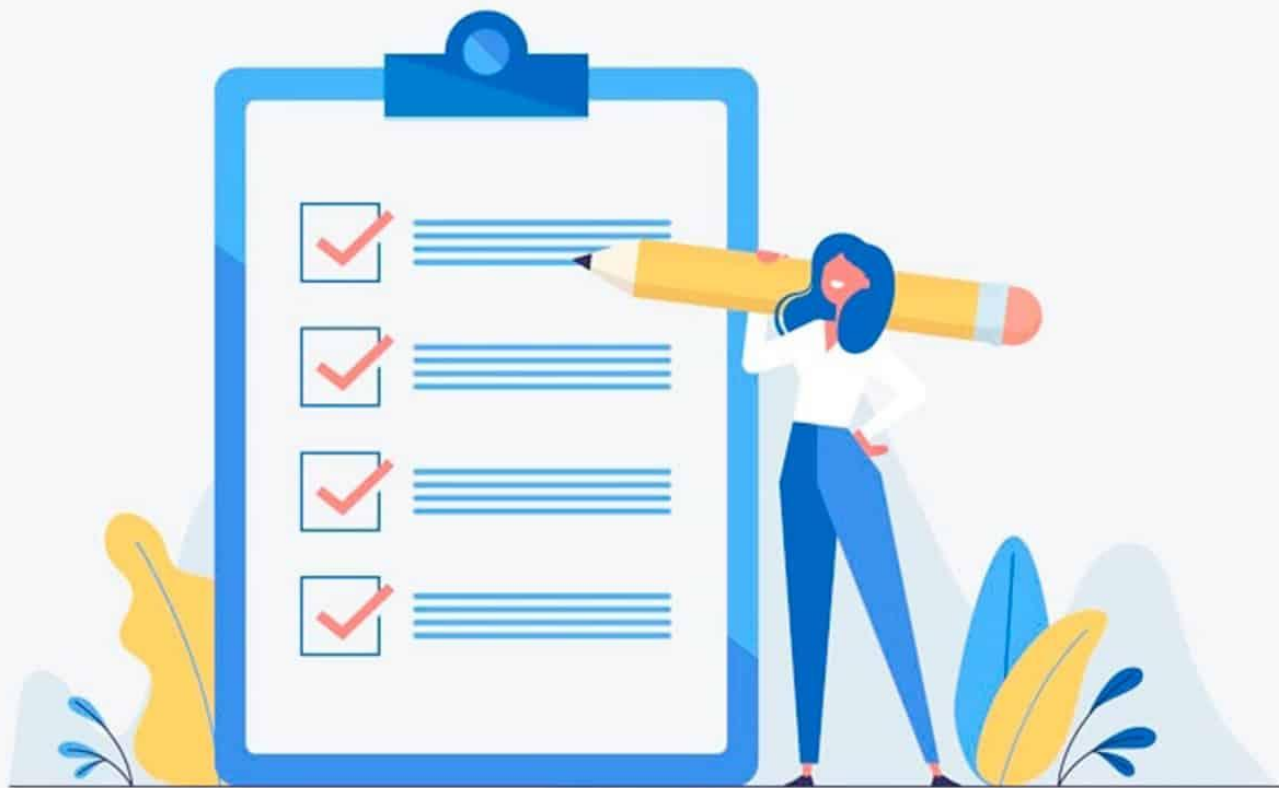
.code segment
start:
; set segment registers:
mov ax,val1
add ax,val2
sub ax,val3
mov valFinal,ax
ends

```

Línea	Verificación
.data segment	Correcta
var1 dw 10000	Correcta
var2 dw 10000	Correcta
var3 dw 10000	Correcta
var4 10000 dw ;instruccion incorrecta	Incorrecta
varFinal dw ?	Correcta
lista BYTE 10,20,30,40	Correcta
lista byte 0ah,20h,22h	Correcta
ends	Correcta
.stack segment	Correcta
dw 128 dup(0)	Correcta

Tabla de Simbolos

Símbolo	Tipo	Valor	Tamaño
var1	variable	10000	WORD
var2	variable	10000	WORD
var3	variable	10000	WORD
varFinal	variable	?	WORD
lista	variable	10,20,30,40	BYTE
lista	variable	0ah,20h,22h	byte



CUESTIONARIO

FUNDAMENTOS DE LENGUAJE ENSAMBLADOR II